

Signal Processing Project

AUDIO WATERMARKING

Authors

Luparu Ioan-Teodor 333

Savin Radu-Andrei 352

Abstract

This paper acts as an introduction to the audio watermarking technique. First it presents fundamental concepts and details important criteria for judging the efficacy of such an algorithm. After that, different methods are presented in detail, as well as various attack methods for destroying hidden information and error correction codes to recover said information. The main purpose of the paper is to showcase the strengths and weaknesses of these algorithms by pitting them against multiple attacks and analysing which ones do better and why.

Contents

1	Introduction	4
1.1	Hiding Information	4
1.2	Audio Watermarking	5
1.3	Design Requirements	5
1.4	Types of Watermarking Methods	6
2	Preliminaries	9
2.1	Audio Watermarking Algorithms	9
2.1.1	Time Domain Methods	9
2.1.2	Transform Domain Methods	11
3	Experiments and Results	15
3.1	Concept	15

Chapter 1

Introduction

1.1 Hiding Information

The rise of the internet has greatly improved the accessibility. People can use the web to easily find, download and share images, music, videos and other forms of digital content. Enforcement of copyright and ownership over such content has become increasingly necessary, but also much more difficult. Various sophisticated techniques have been developed to help content creators protect their work from unauthorized redistribution and piracy and some of them involve the use of steganography.

Steganography is the practice of embedding information within a message or another piece of information such that an unsuspecting individual is not made aware of its presence. In the context of digital information, we typically see a message, image, audio or video hidden within another file that is oftentimes of a different format altogether. Compared to Cryptography, where the existence of a secret message is known or easily detectable but the content is obscured or hard to decode, steganography focuses on concealing the existence of the inserted message.

1.2 Audio Watermarking

Audio is one of the easiest forms of content to share and distribute online, making it a prime target for unauthorized use. To illegally copy an audio file is as easy as recording a playback of it. As such, techniques that aim to protect audio content should utilize the audio signal itself. One of the most popular and effective methods of doing so is through audio watermarking.

Audio Watermarking is a form of audio steganography that involves embedding information into an audio signal. The embedded information, called a watermark, usually contains metadata like the author's name, copyright information or a unique identifier that proves ownership. Audio replay devices or software can then detect such data and identify pirated copies. It is important that audio watermarking algorithms insert watermarks that are difficult to remove or alter without sacrificing the audio quality.

1.3 Design Requirements

When measuring the performance and efficacy of an audio watermarking algorithm, there are several important criteria to consider. These are shown in Figure 1.1 and described below:

- **Imperceptibility:** the watermark must not be noticeable by the human ear. The importance of this criterion is twofold: first, an audible watermark might degrade the listening experience; second, an imperceptible watermark is less likely to be detected and removed by a malicious user.
- **Robustness:** the watermark must be resilient against common audio distortions and be able to withstand weaker audio processing attacks that are designed to not significantly damage the audio quality.
- **Security:** secure watermarking entails using encryption methods to ensure confidentiality. Authorized people are given an extraction key that is needed to retrieve the watermark from the watermarked audio.

- **Capacity:** capacity is measured by how many watermark bits are embedded in a second of the audio. a lower capacity algorithm requires longer audio files to be able to covertly insert longer messages.
- **Computational Complexity:** watermark algorithms should have reasonable computational complexity so that embedding and extraction can be performed efficiently.

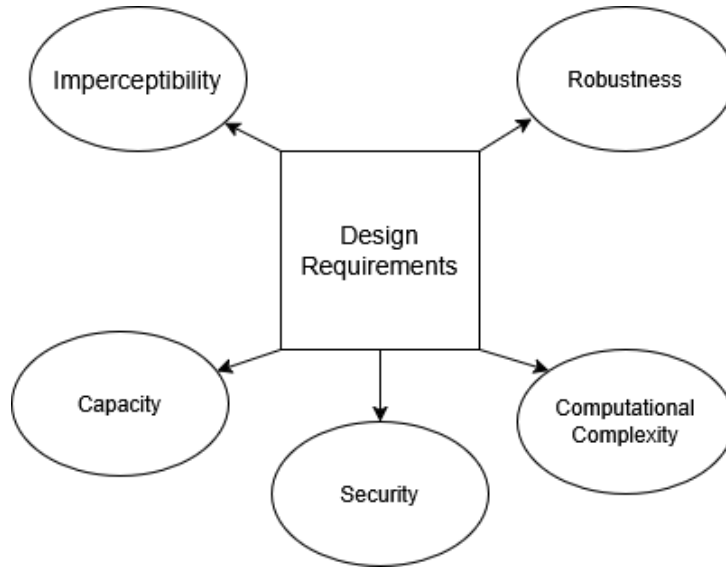


Figure 1.1: Design requirements for audio watermarking algorithms.

When evaluating a watermarking algorithm, it is important to determine how well it balances these criteria, as some of these requirements are often in conflict with one another. For example, a highly robust watermark may require more significant modifications to the audio signal that cause audible distortions, lowering imperceptibility. A well designed algorithm should be robust against common attacks, while maintaining good audio quality and a reasonable capacity and computational complexity.

1.4 Types of Watermarking Methods

Audio Watermarking is not a new field of study. Many different methods have been proposed and researched over several decades. These can be categorised by

the domain in which the watermark is embedded: **Time domain** and **Transform domain**. Figure 1.2 displays categorization for some of the most popular algorithms. Time-Domain watermarking is done by directly modifying the discrete audio samples, while Transform-Domain watermarking involves using transformations to convert the audio signal into a different representation before embedding the watermark there. Figure 1.3 represents a generic audio watermarking system.

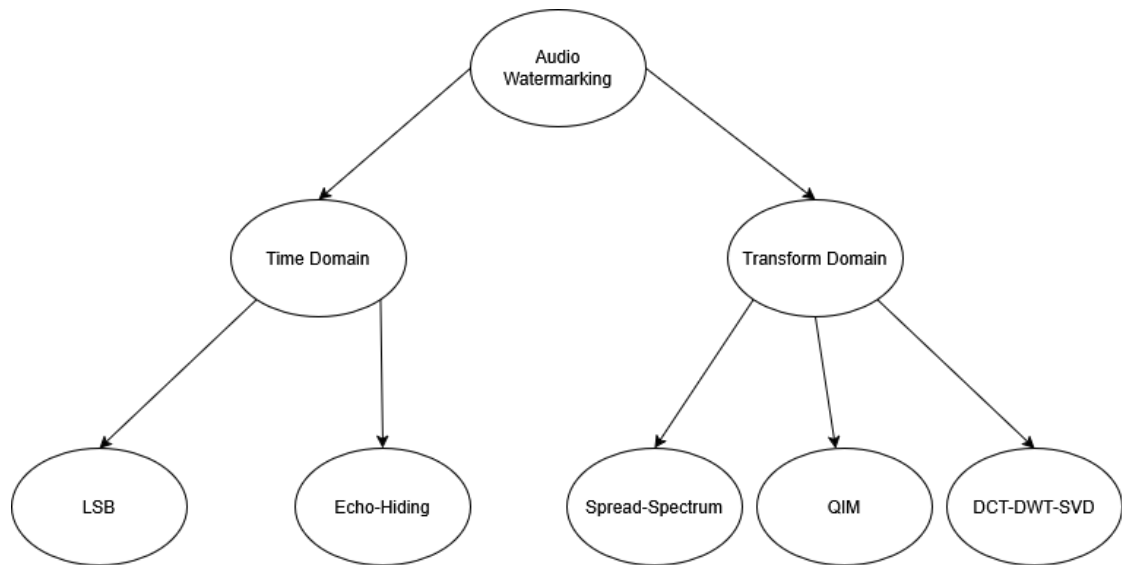


Figure 1.2: Types of audio watermarking algorithms.

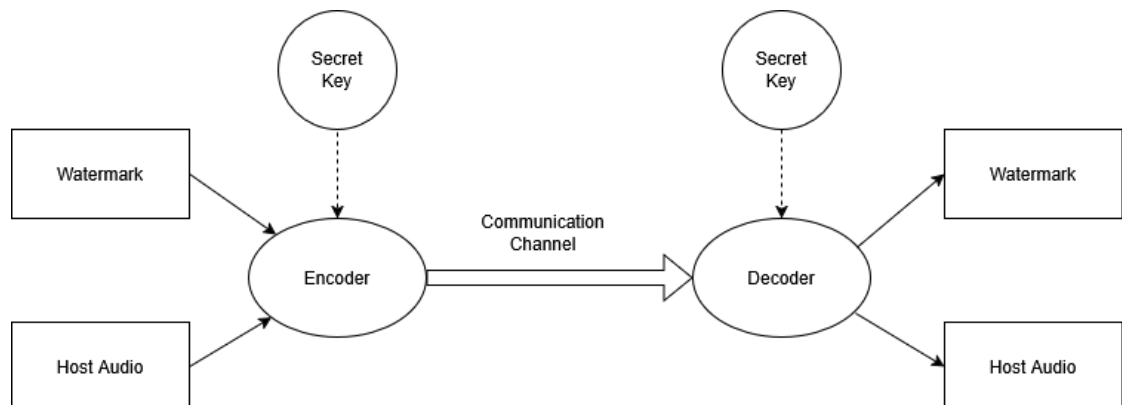


Figure 1.3: Generic audio watermarking system.

Let $x(n)$ note the original host signal in time domain. A typical model for embedding watermark $w(n)$ that produces the watermarked signal $y(n)$ is given by:

$$y(n) = x(n) + \alpha w(n) \quad (1)$$

where α represents the strength of the watermark. A higher α means higher robustness, but also lower imperceptibility. In equation (1) the watermark is added additively, but some methods embed multiplicatively. In these cases equation (1) can be written as:

$$y(n) = x(n) \cdot (1 + \alpha w(n))$$

In transform domain, equation (1) becomes:

$$Y(k) = X(k) + \alpha W(k)$$

where $X(k)$, $W(k)$ and $Y(k)$ are the transformed representations of the original signal, watermark and watermarked signal respectively.

Chapter 2

Preliminaries

2.1 Audio Watermarking Algorithms

In the previous section we discussed different types of audio watermarking algorithms divided into two domain categories. Below we will discuss and explain some of the most popular algorithms from each category that we have implemented and tested for this project.

Every implemented algorithm handles both mono and stereo files. In the case of stereo files, each channel is watermarked independently using the same watermark data for the sake of redundancy for improved robustness. Every algorithm has at least one embedding and one extraction function. Every algorithm has a blind extraction method, meaning it does not need the original audio file to extract the watermark from the modified version.

2.1.1 Time Domain Methods

These methods embed hidden information by directly modifying audio samples. Their decoders detect and extract watermarks by examining how the samples were altered.

Least Significant Bit (LSB) Modulation

This is perhaps the simplest and most intuitive algorithm. Audio samples are represented as binary numbers. The method relies on the fact that the least significant bits of these numbers have minimal impact on audio quality. The algorithm works by replacing the least significant bits of each consecutive sample with bits from the secret message until the entire message has been embedded. Our implementation uses majority voting when modifying more than one LSB per sample.

While this algorithm is very simple and has a high capacity, it has rather poor robustness. Small distortions can easily alter the least significant bits, muddying or even destroying the watermark. For this reason, LSB modulation serves as a good baseline for comparison but is not suitable for real-world applications.

Echo Hiding

This algorithm adds secret messages in the form of echoes of the host audio signal. Information is encoded by modifying delay Δt between the original and the echo. We use delays of Δt and $\Delta t'$ to represent binary '0' and '1' respectively, with both delays being small enough to be imperceptible to the human ear. The basic echo hiding scheme can only embed one bit per echo and as such, instead of using it on the entire audio signal, we must divide it into frames and embed one bit per frame. The base formula for embedding an echo is given by:

$$sw(n) = x(n) + \alpha x(n - \Delta t)$$

where α is the echo amplitude.

We implemented an encoder that uses a decaying kernel which adds multiple echoes with decreasing amplitude for improved robustness. The formula for the general echo-kernel is:

$$h(n) = \delta(n) + \sum_i \left[\underbrace{\overbrace{\alpha_{1,i}\delta(n - d_{1,i})}^{\text{Positive}} - \overbrace{\alpha_{2,i}\delta(n - d_{2,i})}^{\text{Negative}}}_{\text{Forward}} + \underbrace{\alpha_{1,i}\delta(n + d_{1,i}) - \alpha_{2,i}\delta(n + d_{2,i})}_{\text{Backward}} \right]$$

Our encoding function only uses the positive forward echoes for the sake of simplicity.

We first implemented a non-blind extractor that correlates each watermarked frame with the original for both possible delays and decides on the bit whose corresponding delay has a higher correlation. To get consistently good results we also had to subtract the host signal's autocorrelation.

To implement a blind extractor we resorted to using cepstrum autocorrelation. The **cepstrum** is the result of computing the Inverse Fourier Transform of the logarithm of the estimated signal spectrum as shown in Figure 2.1. It is used for investigating periodic structures in frequency spectra, produced in this case by echoes. The cepstrum of a signal containing an echo has an additive periodic component and thus exhibits a peak at each echo offset. In order to get better defined spikes, we applied a window to the watermarked signal before applying the Fast Fourier Transform (FFT), which reduces spectral leakage. The extractor compares peaks at both delays Δt and $\Delta t'$. Not mentioned in the patent we used as reference is that the cepstrum autocorrelation is the Inverse Fourier Transform (IFT) of the power spectrum as stated by the Wiener-Khinchin theorem.



Figure 2.1: Cepstrum computation.

2.1.2 Transform Domain Methods

These methods utilize transformations to change the domain in which the audio signal is represented. The secret message is embedded in this alternate representation before the now watermarked audio is transformed back to the time domain.

In this project we use two popular and important transformations:

- **Discrete Cosine Transform (DCT)**: compared to the Fourier Transform (DFT), DCT only uses cosine functions and as such only yields real-valued coefficients. This makes it computationally more efficient and better suited for audio processing.

- **Discrete Wavelet Transform (DWT):** unlike DCT and DFT, which use sinusoidal basis functions, DWT uses wavelets that are localized in both time and frequency. This allows for improved imperceptibility by focusing on perceptually important components of the audio.

Spread Spectrum

Spread Spectrum is known for its inherent resilience against various signal processing attacks. The main idea behind this algorithm is to spread the hidden information across a broad range of frequencies. The host audio is once again split into frames. Each bit is embedded into the DCT domain. For this, we choose to encode in mid frequencies, as low frequencies more perceptible to the human ear and high frequencies are vulnerable to compression. We also require a pseudorandom (PN) sequence of length $k_2 - k_1$, where k_1 - k_2 is the range of bins in which information is embedded. Consider a single frame $X(k)$ and watermark b together with the spreading sequence $p(k)$. The embedding formula is given by:

$$Y(k) = X(k) + \alpha b p(k), \quad k_1 \leq k \leq k_2$$

where b is 1 if the embedded bit is '1' or -1 otherwise, α is the embedding strength and $X(k)$ is the DCT of the k -th frame. Decoding is done by simply computing the correlation of each frame in the watermarked signal and the same PN sequence used for encoding.

$$C = \langle Y, p \rangle = \langle X, p \rangle + \alpha b \langle p, p \rangle$$

The reason why we choose a pseudo random sequence is because it is statistically orthogonal to the host signal and therefore $\langle X, P \rangle \approx 0$. This entails that $C \approx \alpha b \langle P, P \rangle$ and because we chose $b \in \{-1, 1\}$, C will either be a negative or positive number. This allows us to correctly detect the embedded bit, as $\langle P, P \rangle$ has the same value regardless of the analysed frame from the watermarked signal. Ideally, $\langle X, P \rangle = 0$ but this usually won't be the case. The expanded spread spectrum improves on this by directly subtracting the host signal interference, giving the

following embedding formula:

$$Y(k) = X(k) + (\alpha b - \lambda \Psi)p(k)$$

where $\Psi = \langle X, P \rangle$ and λ is a scaling factor between 0 and 1 that controls how much of the host signal interference is cancelled. The decoder stays unchanged. The main benefits of this improvement is that it makes detection more reliable and allows for choices of smaller alpha (since $\text{Var}(\langle X, P \rangle)$ lowers) which leads to more imperceptible watermarked audio.

Quantization Index Modulation (QIM)

This method entails attributing an unique quantizer to each symbol of the message we wish to embed. For example, a binary message would require only two quantizers. These quantizers are used to alter the values of the DCT coefficients of the signal such that a decoder can tell which one was used over a set of consecutive values in order to correctly estimate the corresponding message symbol. By aiming for middle frequencies and changing coefficients by small amounts, we can ensure good imperceptibility.

Our QIM encoder groups audio samples into frames of the same size and enumerates through the symbols of the watermark in bits form. For each bit in the message, we take the DCT coefficients of a frame and snap them to the nearest value of the bit's specific quantizer. Since our watermark is in binary, we have Q0 for bit 0 and Q1 for bit 1 with the quantizers being offset versions of each other (For step size = 10, we have Q0 = 0, 10, 20, 30, ... and Q1 = 5, 15, 25, ...). We also utilize frame overlapping in order to prevent abrupt changes between frames when we have alternating bit values. The decoder goes through the same frames and estimates which quantizer was likely used to decide the current bit's value.

In addition to the basic QIM algorithm, we have found and implemented two improved versions:

- **Dither:** QIM-Dither rectifies a problem produced by the way information is embedded. The QIM algorithm may produce periodic distortion patterns in the watermarked signal, which can be exploited by an attacker to remove

the watermark. To prevent this, a small random value called a dither is added to each coefficient before quantization. This randomizes the distortion patterns and makes it more difficult for an attacker to identify and remove the watermark.

- **DWT:** This version of QIM uses the DWT instead of the DCT when transforming the host signal. This allows for better imperceptibility and robustness by taking advantage of the time-frequency localization properties of wavelets.

DCT-DWT-SVD Algorithm

This algorithm was proposed in [] and combines the DCT and DWT transformations with Singular Value Decomposition (SVD) to achieve a robust and imperceptible watermarking scheme. The previously mentioned benefits of DCT and DWT are leveraged alongside many useful properties of SVD such as stability, scalability and proportionality. This algorithm embeds watermark data into the highest singular values of DWT-DCT transformed audio frames. This assures robustness as singular values are much less affected by common signal processing attacks.

While this initial version only includes a non-blind extractor, we have modified the algorithm to allow for blind extraction by using the predefined quantization coefficient Q in a similar manner to QIM. The additional predefined constants $K1$ and $K2$ are not used in our implementation.

Chapter 3

Experiments and Results

3.1 Concept

We conducted experiments on a dataset of 20 audio excerpts of equal duration from different Minecraft songs. First we show the robustness vs. attack strength graphs, as described on page 113 of [book]. They show the bit error as a function of the attack strength for a fixed audio quality. For each algorithm and attack strength, watermark extraction was performed on all test signals, and the resulting bit error rates were averaged. Embedding parameters were selected such that all watermarking algorithms produced comparable perceptual audio quality, as measured by Objective Difference Grade (ODG). This metric's values range from 0 (imperceptible difference) to 4 (very annoying distortion). More details about it can be found in []. Each algorithm was subjected to a different suite of attacks SNR was also reported for reference. All evaluated watermarking schemes perform blind extraction but are not synchronization-invariant. Therefore, attacks involving time-domain editing, such as cropping or segment replacement were excluded as they would disrupt watermark synchronization and may lead to complete decoding failure. We still used a form of cropping attack that replaces a contiguous segment of the watermarked audio with the corresponding segment from the original (unwatermarked) signal, as is utilised in [SVD paper].

We also used a subjective testing measure known as the AXB test (III A of the article). The subject is presented with the audio signals A and B (the original and

watermarked audio signals in random order) and the audio signal X (the original audio signal), and is tasked with guessing which between A and B is most similar to X. We tested 3 different people and got the following results: